of starting test method names with the word "test." For example, the following JUnit tests follow this convention:

```
@Test
public void testAddStatusPoints() {...}

@Test
public void testTransferPoints() {...}
```

Some earlier unit-testing libraries required you to respect this convention, and many development tools still encourage it by letting you generate a unit test for each method in your class, prefixed with the word "test." And this convention does make it easy to see where a particular method has been tested.

But from a BDD perspective, this approach has some limitations. First, it assumes that the method under test already exists, or that you know what it should be called from the outset. It also binds a test to a particular method, which limits the ways you might want to test a given method and makes for more work when refactoring.

In BDD you focus on specifying the *behavior* of a class, rather than just testing its methods. After all, the methods are simply a way of getting the class to perform some particular task or behavior. To reflect this, instead of naming a test after the method it's testing, you name the test based on what you expect the class to do. For example, in .NET you might write tests like the following:

```
[TestFixture]
public class WhenUpdatingStatusPoints
{
    [Test]
    public void ShouldBeAbleToAddStatusPointsEarnedFromAFlight()
    {...}

    [Test]
    public void ShouldUpdateStatusWhenEnoughStatusPointsAreEarned()
    {...}
}
```

These test names are designed to be part of the living technical documentation, so clarity and readability are key. Some Java and .NET practitioners prefer to use a more readable notation for test names based on underscores instead of CamelCase. For example, in JUnit you could write the following:

```
@Test
public void should_be_able_to_add_status_points_earned_from_a_flight() {
    ...
}

@Test
public void should_update_status_when_enough_status_points_are_earned() {
    ...
}
```

Other practitioners use a slight variation on this approach that respects the use of the "test" prefix. Instead of using the word "should," they use "test that." This works well